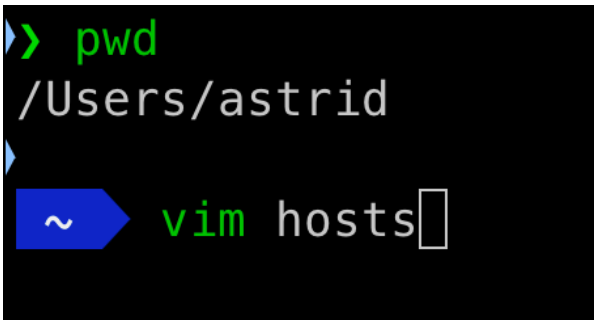
Module 15 - chapter 4

Ansible Inventory and Ansible ad-hoc commands

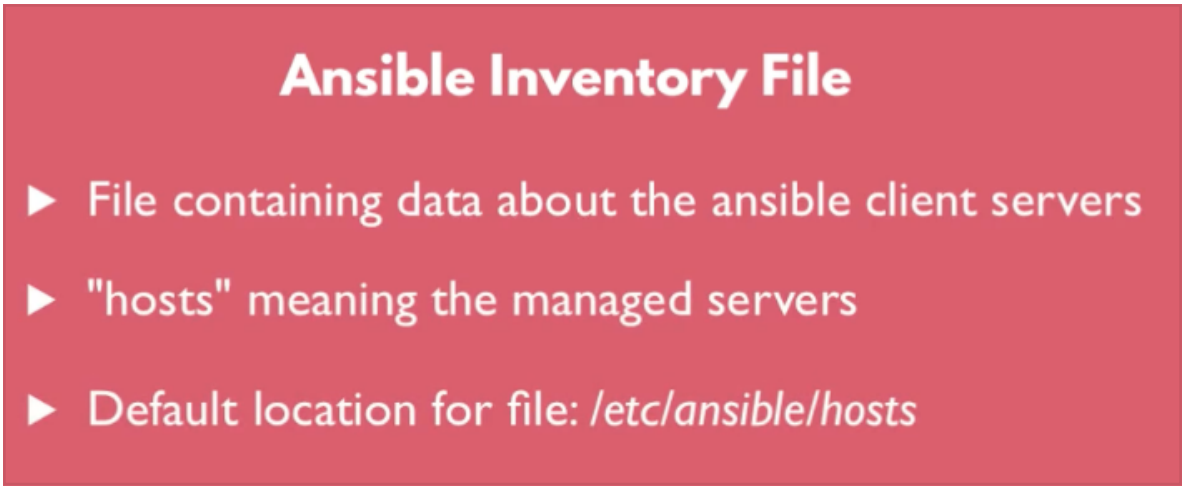
Now we will connect Ansible to the two remote target servers from our laptop the control machine.

1. Tell Ansible the IP addresses or hostnames of the servers that it has to manage -> we use a file called ‘hosts’ that we need to create:

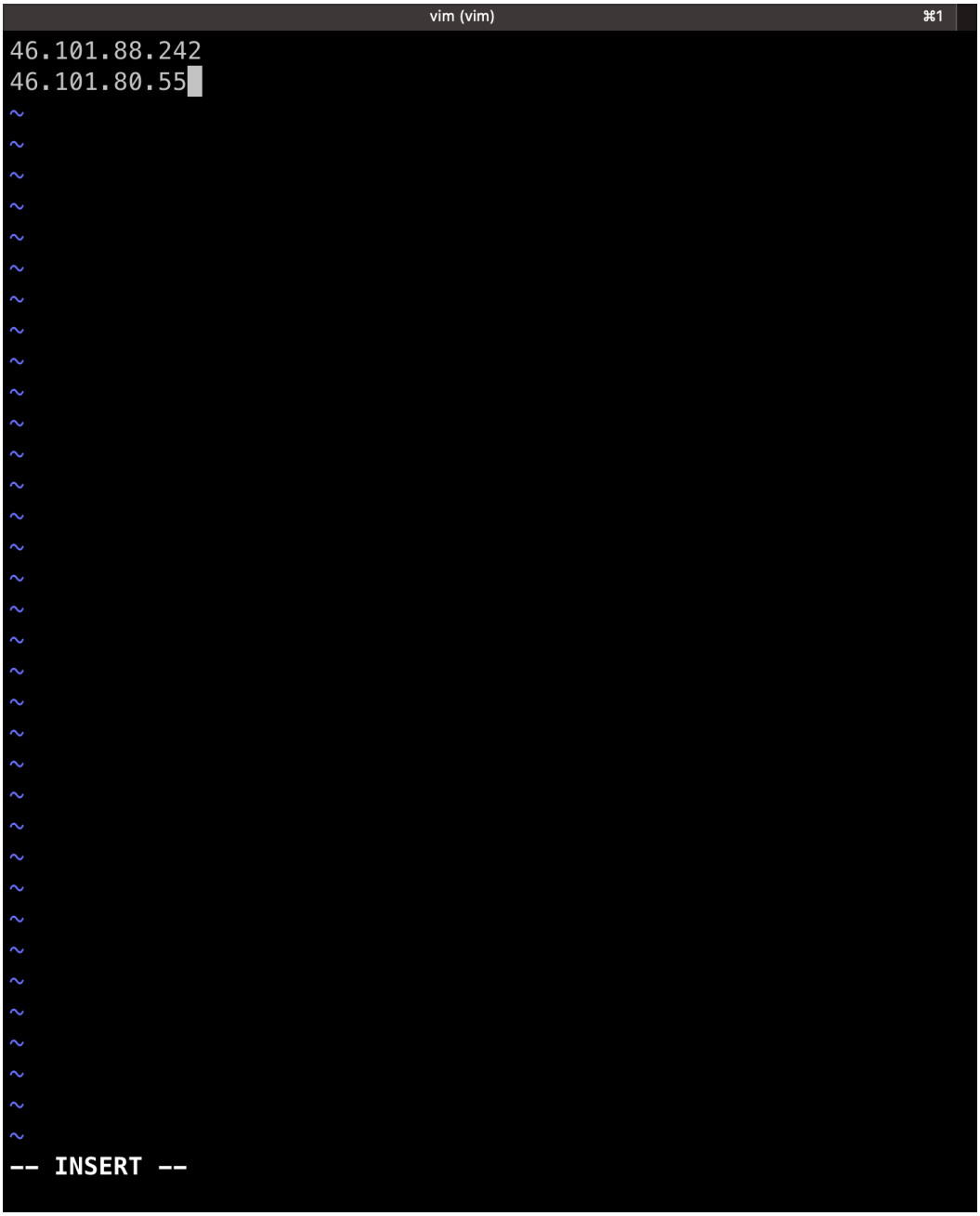


-> vim hosts

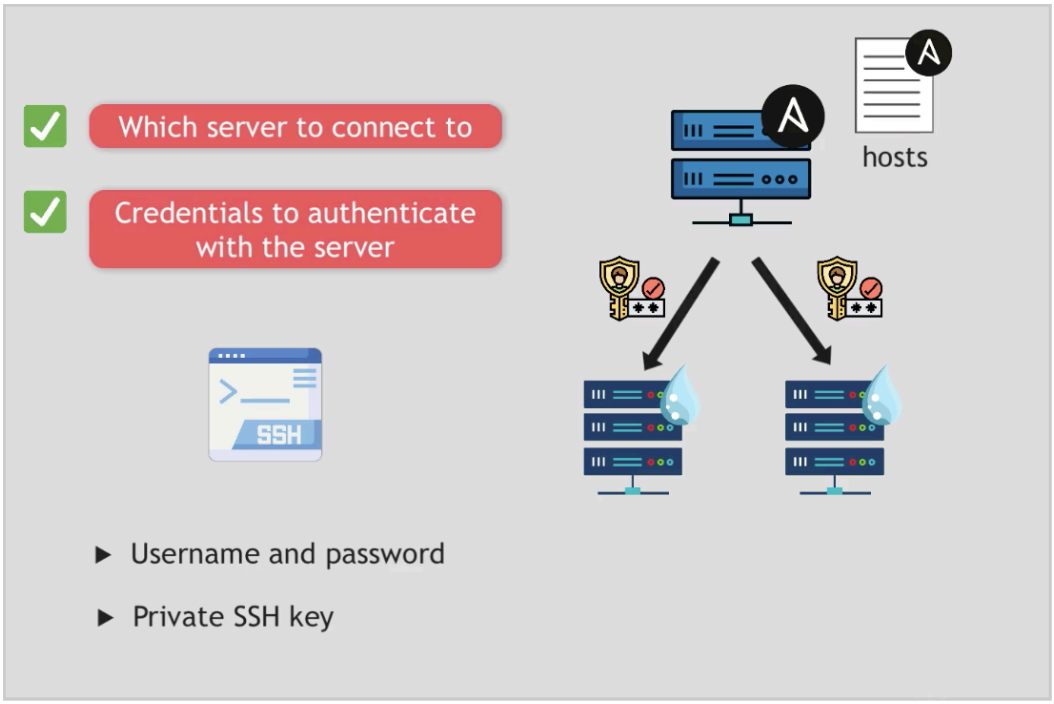
And now we write the Ansible INVENTORY



So I add the two ip addresses to the file



And now Ansible will know the servers that it needs to connect to



And Ansible needs credentials to authenticate with the server via SSH, and it can be done in two ways:

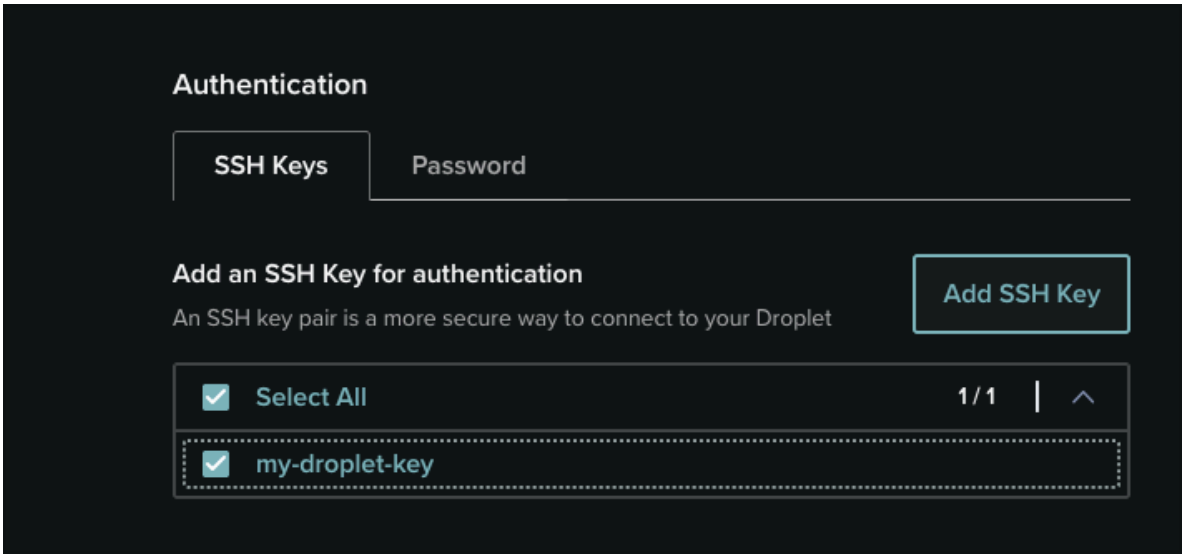
- Username and password
- Private SSH key

We will use the private SSH key way.

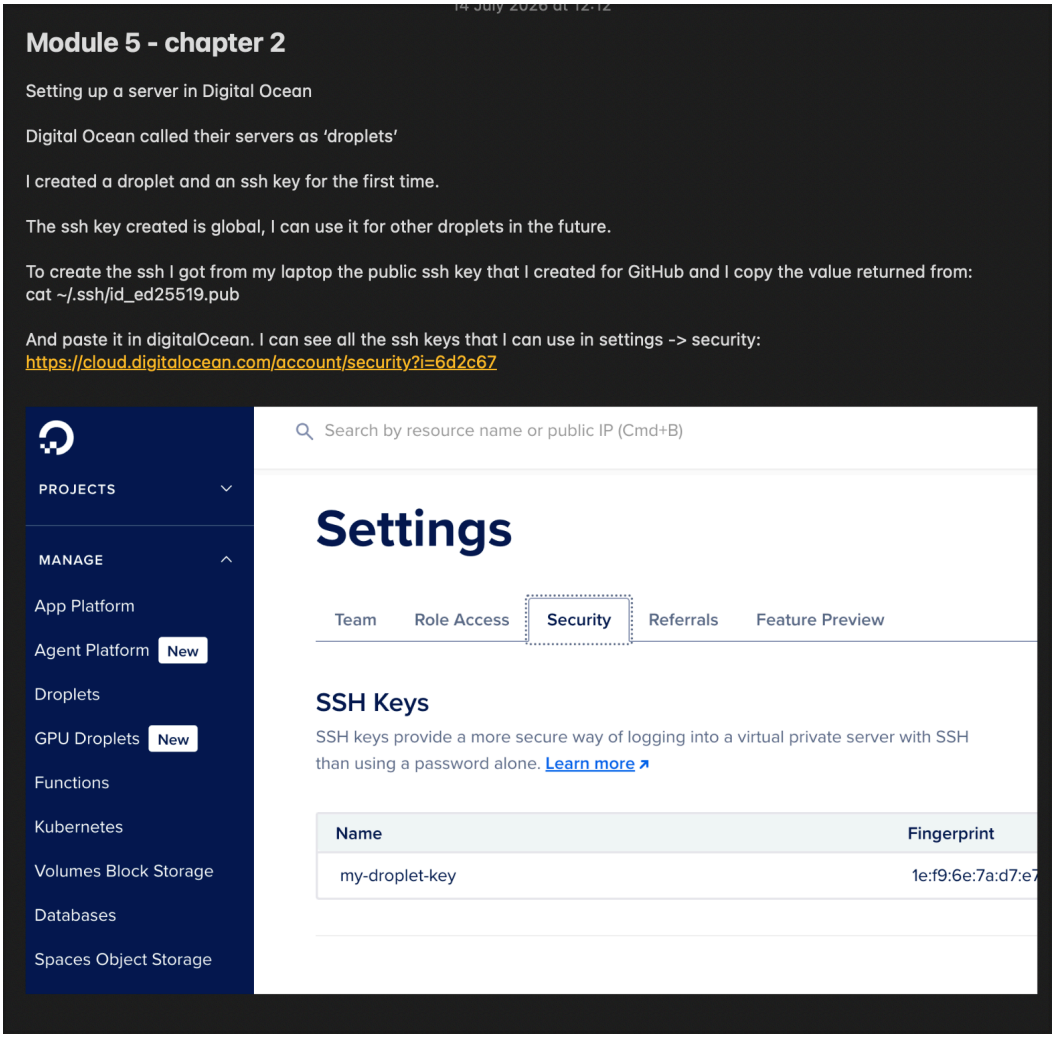
And in this ‘hosts’ file we can specify which private SSH key to use for each host -> using an attribute called
‘ansible_ssh_private_key_file’

In our case our ssh key is stored in our .ssh folder so:
-> ansible_ssh_private_key_file=~/.ssh/id_id_ed25519

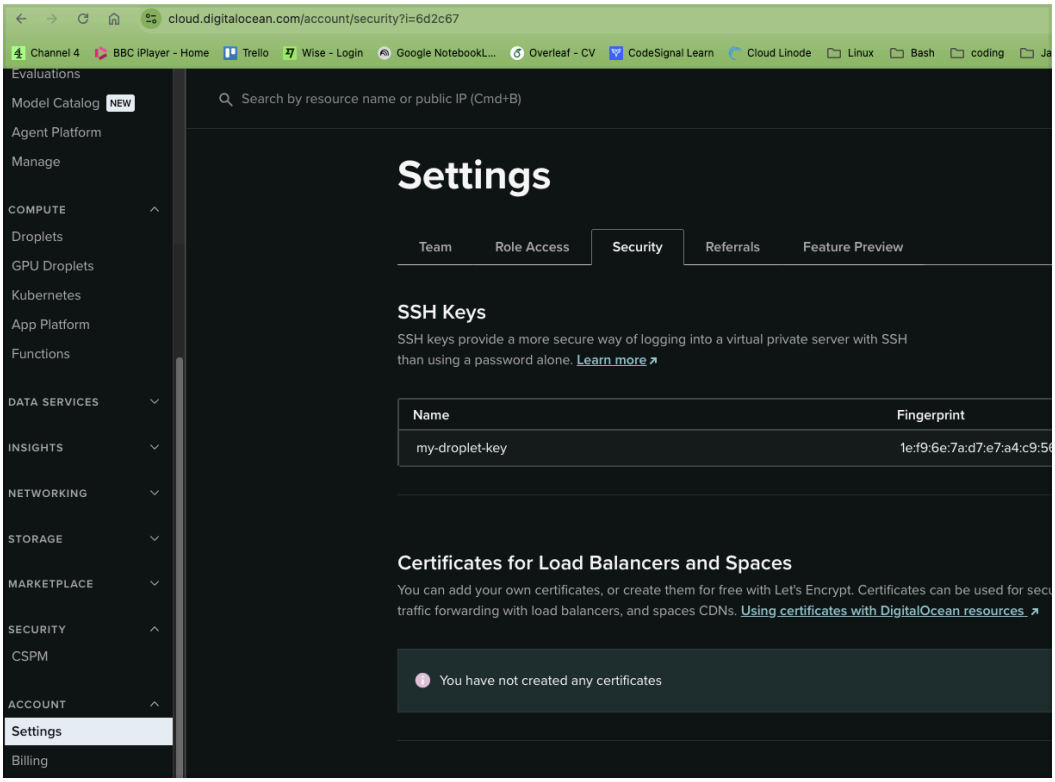
And remember that when creating the droplets we selected the SHH key ‘my-droplet-key’



Here part of the notes when I created the ssh key in DigitalOcean from Module 5 - chapter 2

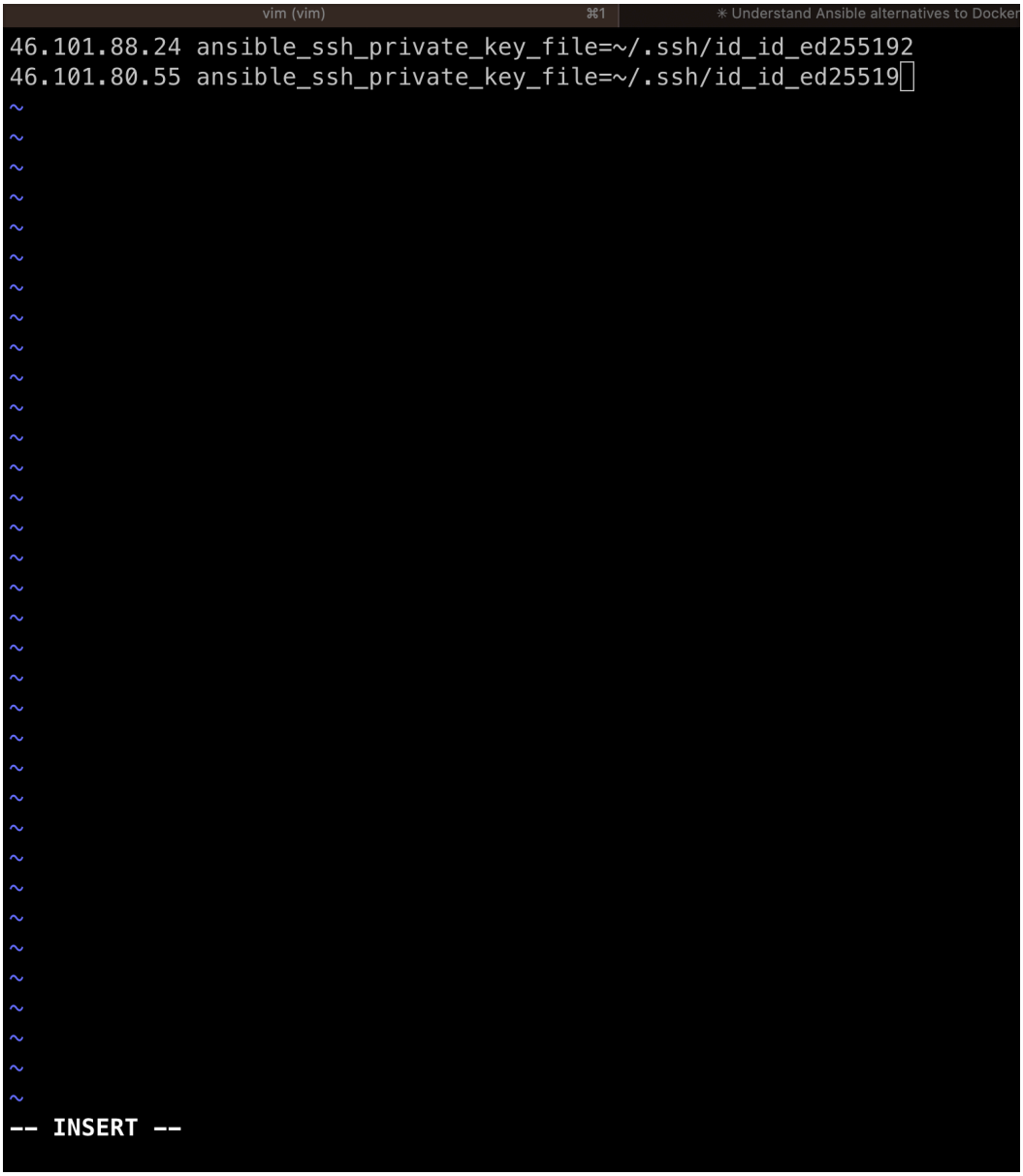


And here we can see the SHH info



So the SSH public key (id_ed25519.pub) is already set in DigitalOcean and our servers are using it.

Now in the ‘hosts’ file we pass the private key -> ‘id_ed25519’



So we have the IP address and the credential for Ansible to reach each target server and have access.

When I as a human ssh into the server I write (uses default private key so no need to type it)
-> ssh root@<IP_Address>

Or I specify the private key location with if is different to the default private key:
-> ssh -i ~/.ssh/id_ed25519 **root**@<IP_Address>

'Hosts' already have <IP_Address> and the location to the private key ~/.ssh/id_ed25519 so we are missing the user, in this case 'root' and for that we use the attribute 'ansible_user'
-> ansible_user=root

And I safe the file and I check it:

Now we can test whether ansible can connect to the 2 servers, using Ansible ad-hoc commands -> a one-off task run directly from the CLI, not a full playbook.

-> ansible all -i hosts -m ping

Part	Meaning
`ansible`	The ad-hoc command-runner CLI (different from `ansible-playbook`, which runs `.yml` files)
`all`	The **target pattern** — which hosts to run against. `all` means every host defined in the inventory. Could instead be a group name (e.g. `webservers`) or a single hostname.
`-i hosts`	The **inventory file** to read hosts/groups from — a file named `hosts` in the current directory, listing IPs/hostnames (optionally grouped, e.g. `[webservers]` sections)
`-m ping`	The **module** to run — Ansible's `ping` module

The important gotcha: ping here is not ICMP ping

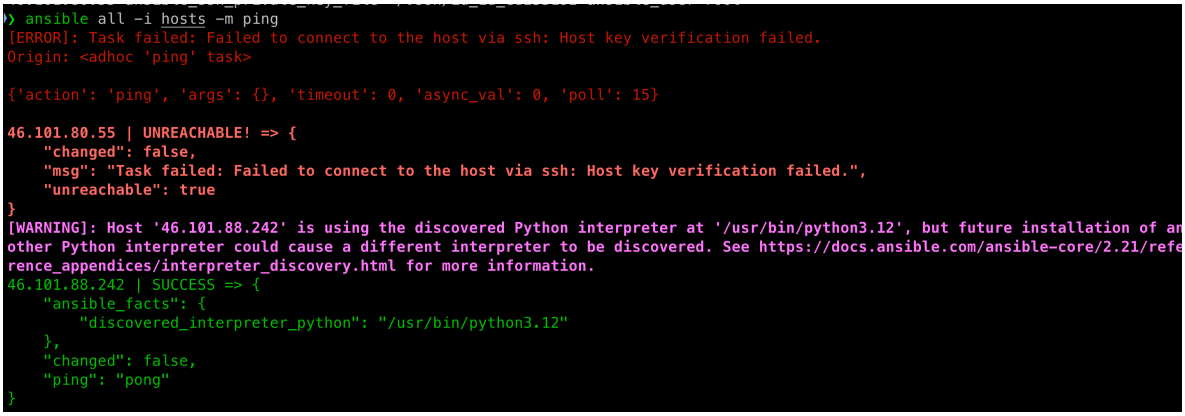
Ansible's ping module does not send ICMP packets like the ping command in your terminal. Instead it:
1. Connects to each target over SSH
2. Checks Python is available on the remote machine
3. Returns "pong" if successful

So this command is really: "Can I reach every host in my inventory over SSH and run Python on it?" — it's the standard first sanity check people run after writing an inventory file, before trying any real playbook, because it validates connectivity + auth + the Python prerequisite all in one shot.

Example output for a working host:
```
web1.example.com | SUCCESS => {
    "changed": false,
    "ping": "pong"
}
```

If it fails, the error usually points to SSH key/auth issues or a missing Python interpreter on the target — exactly the kind of thing you'd want to catch before running an actual provisioning playbook against it.

So let’s run the ad-hoc command



And one server fails, this is because the HOST hey verification failed which is a separate problem from authentication (your SSH key itself is fine, since the other host, 46.101.88.242, connected and even ran a ping successfully using the same key file).

So ‘hosts’ file is correct.

What's actually happening

Every SSH server has a host key (its identity fingerprint). The first time you connect to a server, SSH records that fingerprint in ~/.ssh/known_hosts. On every future connection, SSH compares the fingerprint the server presents against what's stored — if they don't match (or nothing was ever stored, in a non-interactive context like Ansible), SSH refuses to connect. This exists to protect against man-in-the-middle attacks — it's SSH asking "is this really the same server I trusted before?"

Two likely causes for 46.101.80.55 specifically:

1. You've never SSHed into it interactively before, so there's no entry in known_hosts yet. A normal interactive ssh session would prompt you "Are you sure you want to continue connecting (yes/no)?" — but Ansible runs non-interactively, so it can't answer that prompt and just fails closed instead.
2. The server's host key changed since you last connected — very common with cloud VPS providers (this IP range, 46.101.x.x, is a DigitalOcean block) when a droplet gets destroyed and recreated reusing the same IP. In that case, ssh would normally show a loud REMOTE HOST IDENTIFICATION HAS CHANGED! warning rather than the quieter "never seen this host" case.

An I know I have not SSHed into that server yet (I have SSHed into the other server, so the ~/.ssh/known_hosts has the info of that server but not the other).

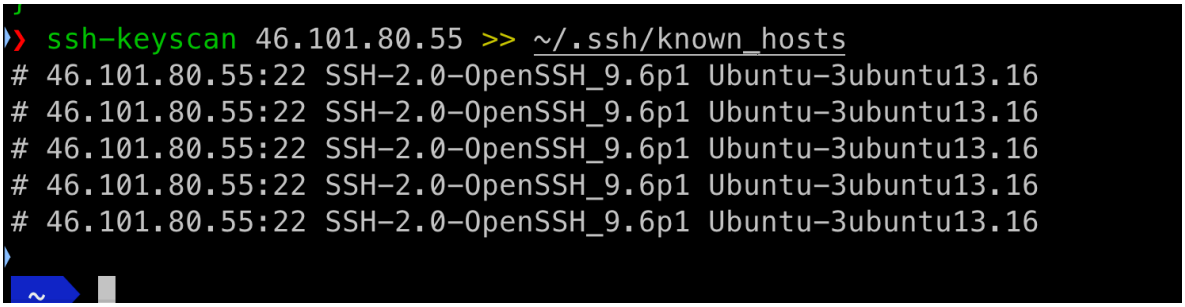
So the server's host key was never trusted locally, so Ansible's non-interactive SSH refuses to proceed without a prompt to answer.

Two ways to fix it:

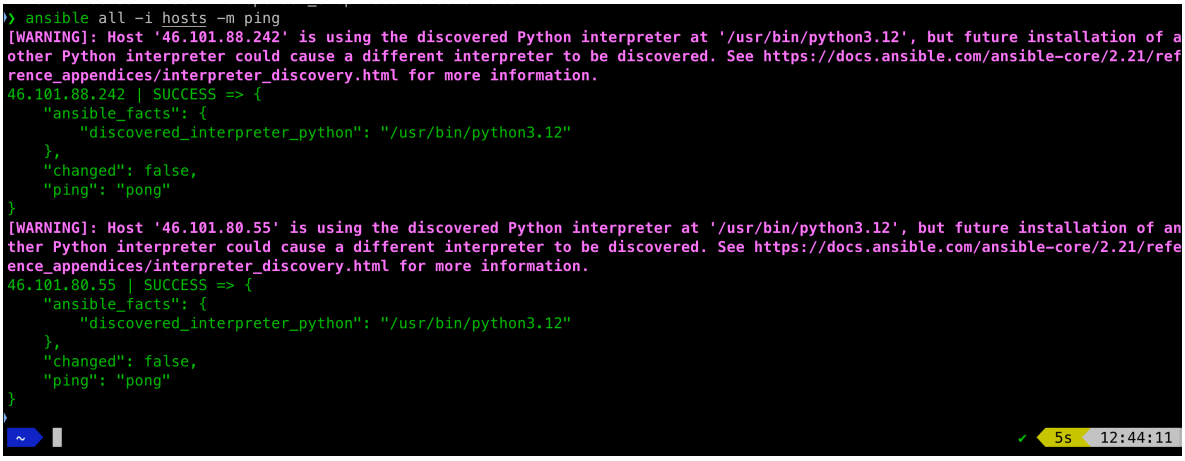
Option A — trust it in advance (recommended, no manual prompt needed):
ssh-keyscan 46.101.80.55 >> ~/.ssh/known_hosts
This fetches the server's host key and adds it to your trusted list, same end state as answering "yes" interactively — just non-interactive.

Option B — SSH in manually once, accept the prompt:
ssh -i ~/.ssh/id_id_ed255192 root@46.101.80.55
Type yes when asked, then exit. Future Ansible runs will work.

So I run option A
-> ssh-keyscan 46.101.80.55 >> ~/.ssh/known_hosts



And try the ad-hoc Ansible command again



And it works!

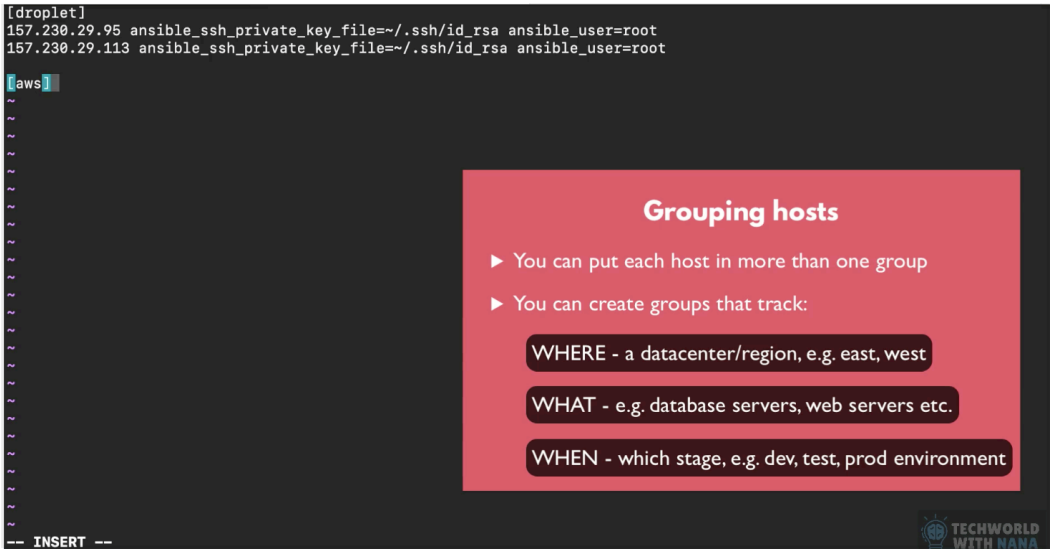
Now let’s imagine we have servers in different platforms

So when running
-> ansible all -i hosts -m ping

all -> not always will work as some servers belong to one platform and others to a different platform hence having different requirements or execute different commands to access them.

So what we do is to group the servers.
By adding a name at the beginning of the group inside square brackets -> [droplet]

And if we had AWS instances it will look like this:



Another example is instead of grouping by platform we can group by type, like having 5 servers that are for databases and 5 servers for web application, so:

```
[database]
157.230.29.95 ansible_ssh_private_key_file=~/.ssh/id_rsa ansible_user=root
157.230.29.113 ansible_ssh_private_key_file=~/.ssh/id_rsa ansible_user=root

[web]
~
~
~
```

For databases we will perform different operations (commands) than for web applications so executing ‘all’ won’t work as each group has different operations. but now we have the groups and we can use their names to deal with the differences.

And now this is what I have:

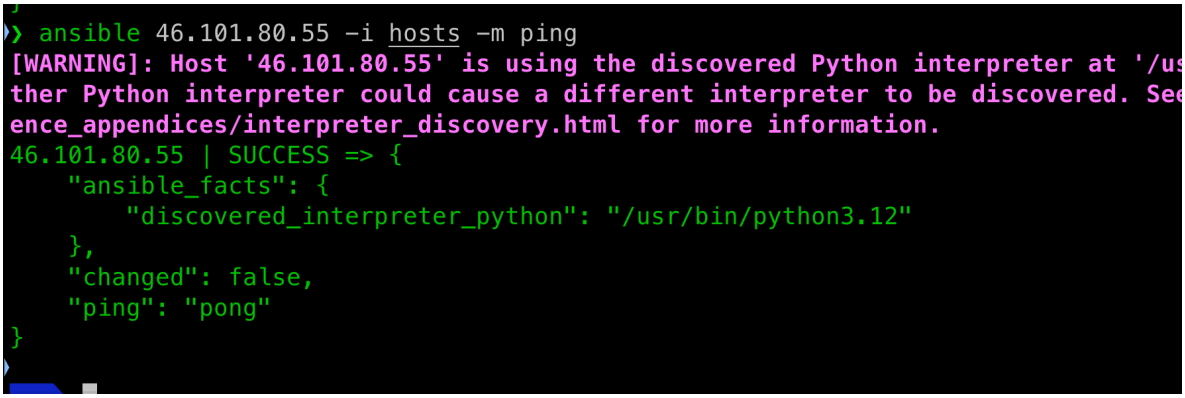
```
❯  vim hosts
❯ cat hosts
[droplet]
46.101.88.242 ansible_ssh_private_key_file=~/.ssh/id_id_ed255192 ansible_user=root
46.101.80.55 ansible_ssh_private_key_file=~/.ssh/id_id_ed255192 ansible_user=root
❯
```

Now we can execute ad-hoc ansible:
-> ansible droplet -i hosts -m ping

droplet -> group name instead of ‘all’

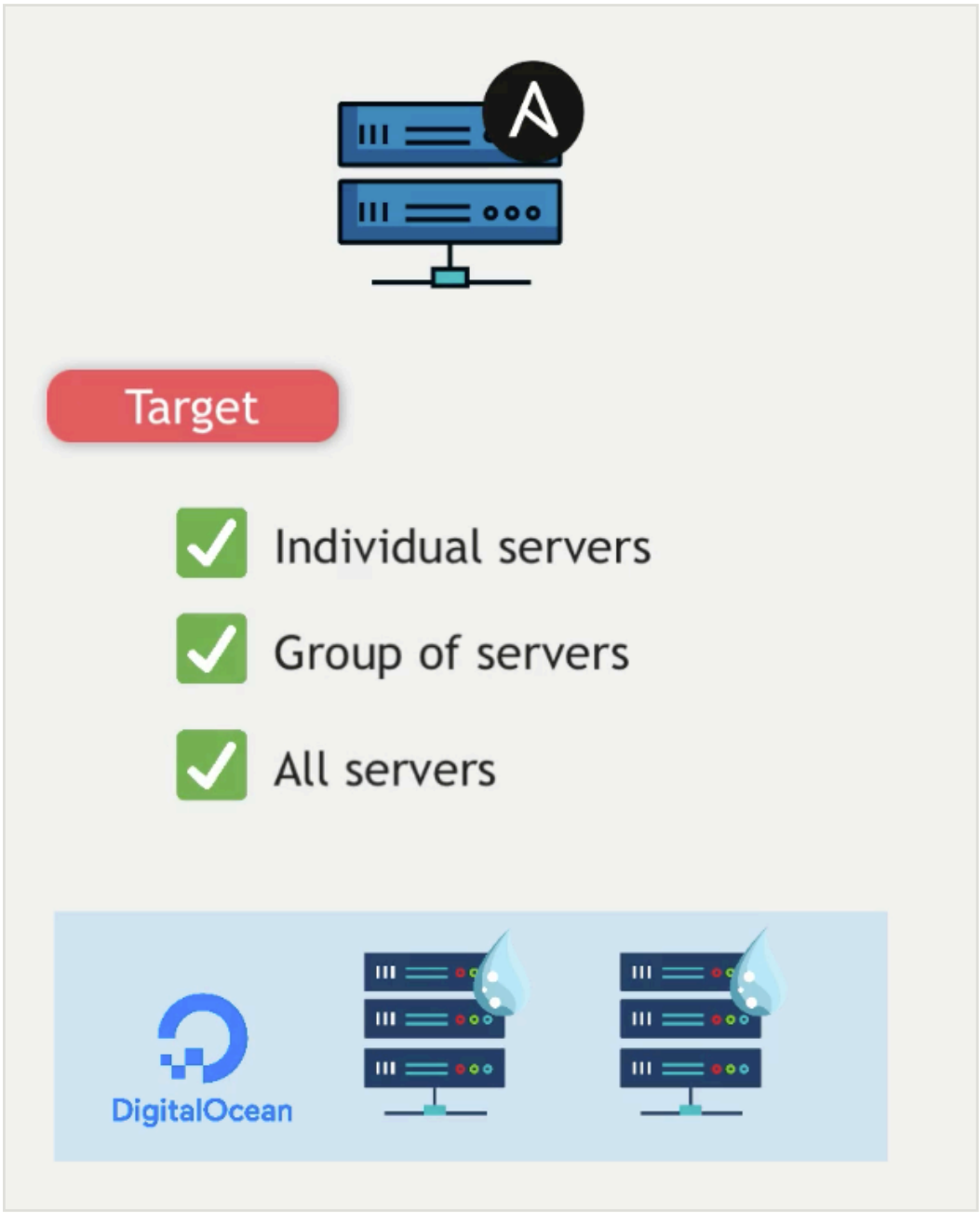
```
❯ ansible droplet -i hosts -m ping
[WARNING]: Host '46.101.80.55' is using the discovered Python interpreter at '/usr/bin/python3.12', but future instal
ther Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-co
ence_appendices/interpreter_discovery.html for more information.
46.101.80.55 | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3.12"
    },
    "changed": false,
    "ping": "pong"
}
[WARNING]: Host '46.101.88.242' is using the discovered Python interpreter at '/usr/bin/python3.12', but future insta
other Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.com/ansible-c
rence_appendices/interpreter_discovery.html for more information.
46.101.88.242 | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3.12"
    },
    "changed": false,
    "ping": "pong"
}
```

If I want to execute the command only for one specific server then I provide the IP address
-> ansible 46.101.80.55 -i hosts -m ping



So with ansible we can target:

- Individual servers
- Or a group of servers
- Or all of the servers



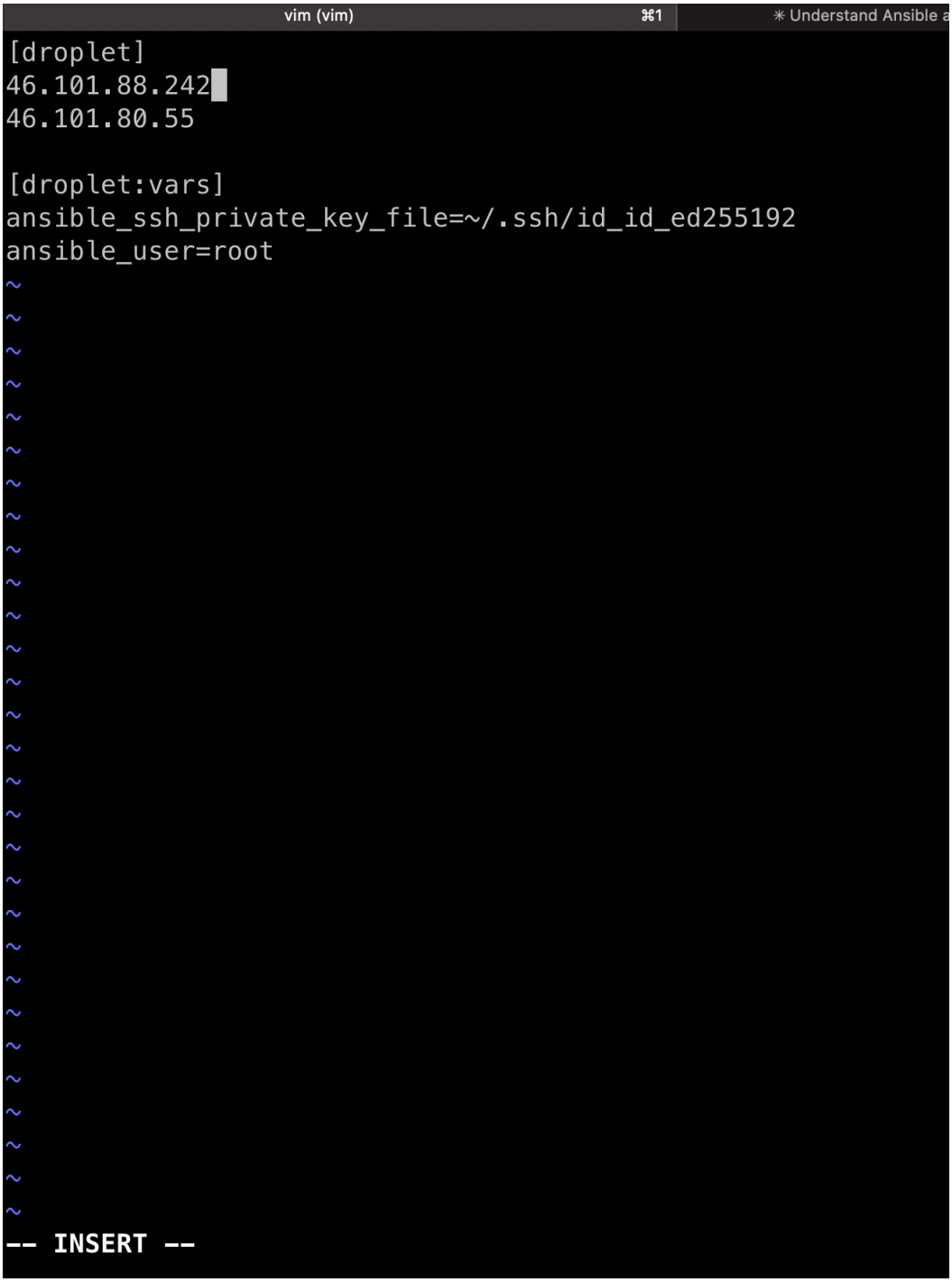
Now there is another improvement we can do to the hosts file. If we have 50 servers with the same ssh key and same user, we will be repeating the same attributes 50 times and is not efficient so what we can do is give those values to the group as variables.

We move the values to the vars

```
[droplet]
157.230.29.95 ansible_ssh_private_key_file=~/.ssh/id_rsa ansible_user=root
157.230.29.113 ansible_ssh_private_key_file=~/.ssh/id_rsa ansible_user=root

[droplet:vars]
ansible_ssh_private_key_file=~/.ssh/id_rsa
ansible_user=root
~
~
~
```

And now it looks like this:



So now we have the list of servers in a group.
And we also have the configuration of the group using the attributes.

And we test again

```
❯ vim hosts
❯ cat hosts
[droplet]
46.101.88.242
46.101.80.55

[droplet:vars]
ansible_ssh_private_key_file=~/.ssh/id_id_ed255192
ansible_user=root
❯ ansible droplet -i hosts -m ping
[WARNING]: Host '46.101.80.55' is using the discovered Python interpreter at '/usr/bin/python3.12', but
ther Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible.
ence_appendices/interpreter_discovery.html for more information.
46.101.80.55 | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3.12"
    },
    "changed": false,
    "ping": "pong"
}
[WARNING]: Host '46.101.88.242' is using the discovered Python interpreter at '/usr/bin/python3.12', bu
other Python interpreter could cause a different interpreter to be discovered. See https://docs.ansible
rence_appendices/interpreter_discovery.html for more information.
46.101.88.242 | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3.12"
    },
    "changed": false,
    "ping": "pong"
}
```